HTTP requests. The file exports the `MovieAPI` so the GraphQL server can use the `getMovies` and `getMovieById` functions.

AN OVERVIEW OF GRAPHQL-SERVER.JS

Within the provider folder, there is another class named graphql-server.js, which represents the code to set up our GraphQL server. Let's break down the code into different parts:

```
const { ApolloServer } = require('apollo-server');
const { startStandaloneServer } = require('@apollo/server/standalone');
const MovieAPI = require('./data-source.js');
const gql = require('graphql-tag');
```

First, we import all the required packages needed to set up our GraphQL server. The `apollo-server` package is an open source library for server-side GraphQL, which exports the `ApolloServer` class.

> **Note** We're using Apollo Server 4 for this project. If you're familiar with Apollo Server 3, there are significant changes between the two versions, which we won't cover as part of this book. However, if you're interested in migrating from Apollo Server 3 to 4, then check out this useful migration guide from Apollo at www.apollographql.com/docs/apollo-server/migration.

Because we're using Apollo Server 4, we also need to export the `startStandaloneServer` function from `@apollo/server/standalone`, which is a recommended change. We're also exporting the `MovieAPI` for our data source and the `gql` template literal tag from `graphql-tag`, a helpful utility for parsing GraphQL queries and defining our GraphQL schema. Next, we define our GraphQL schema: